

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

SEMESTER PROJECT SPRING 2026

MASTER IN APPLIED MATHEMATICS

Robust High-Dimensional Regression: Beyond i.i.d. Noise

Author:
Mohamed Ali MOUTIE

Supervisors:
Dr. Hossein GORJI
Ramzi DAKHMOUCHE

EPFL

Abstract

Regression under i.i.d. noise is well understood, but this assumption rarely holds in practice. Measurement pipelines, numerical PDE solvers, and privacy constraints all induce noise that is correlated, heavy-tailed, or structured at the batch level. In the worst regimes, even the correspondence between an input and its observed output is lost, and it is precisely here that losses like the MSE break down, since they assume a fixed pointwise input-output match. In this project, we study the Wasserstein Batch Matching (WBM) framework of Dakhmouche et al. (2025) [1], which relaxes this match into a distributional, batch-level one driven by the Wasserstein distance, and we extend it in three directions. First, we derive the influence function of the WBM estimator, and show that in the linear case it is not B-robust: its influence grows linearly in the contamination point. This suggests that the strength of WBM does not come from bounding the effect of outliers, but rather from its tolerance to a broken input-output correspondence. We then illustrate this on low-dimensional examples, where, under batch-level shuffling, the MSE collapses towards the mean while WBM still recovers the global shape of both a 1D and a 2D target. Finally, we apply WBM to operator learning on the Navier–Stokes equations. Under batch-level shuffling, WBM outperforms the L^1 baseline at nearly every corruption rate, with a near two-fold error reduction at $\rho = 0.75$. Under heavy-tailed Cauchy corruption, however, its advantage holds only at low corruption and reverses once corruption is severe, which is consistent with what the non-B-robustness of the estimator predicts. Together, these results help pin down where it shines: WBM is robust to corruption that preserves the batch output distribution, and to heavy tailed noise in settings of moderate corruption.

Contents

1	Introduction	1
1.1	The classical framework	1
1.2	Non-iid noise models	1
1.3	WDRL vs WBM (Dakhmouche et al. 2025)	3
1.3.1	WDRL fails for non-convex/non-lipschitz model.	3
1.3.2	Wasserstein Batch Matching (WBM)	3
2	First Experiments	3
2.1	WBM vs MSE without shuffling: fitting 1D function	3
2.2	WBM vs MSE with shuffling: fitting 1D function	5
2.3	WBM vs MSE with shuffling: fitting 2D surface	5
2.4	Takeaway	7
3	Interlude on Influence	7
3.1	Influence function of $\hat{\theta}_{\mathbf{WBM}}$	7
3.2	The linear case	8
4	WBM applied to operator learning	10
4.1	Operator Learning: A quick overview	10
4.2	Convolutional Neural Operators	11
4.2.1	Neural Operator	11
4.2.2	CNOs	11
4.3	Learning Navier Stokes	11
5	Conclusion	13

1 Introduction

Regression under the assumption of i.i.d. noise is well understood, but this assumption rarely holds in practice: measurement pipelines, numerical solvers, and privacy constraints all induce noise that is correlated, heavy-tailed, or structured at the batch level. This report studies the Wasserstein Batch Matching framework of [1] as a response to such regimes. We first review the classical setup and the noise models that break it (§1), reproduce WBM on low-dimensional benchmarks (§2), derive the influence function of the WBM estimator (§3), and apply the method to operator learning on the Navier–Stokes equations (§4).

1.1 The classical framework

Consider the following model:

$$Y_i = f^*(X_i) + \varepsilon_i, \quad \varepsilon_i \sim F$$

given F a probability distribution. Recall the classical Empirical Risk Minimization framework that we usually consider in ML contexts, where we minimize the following objective function:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \|Y_i - f_{\theta}(X_i)\|^2$$

for some parametric family of models $\{f_{\theta}, \theta \in \Theta\}$.

One can show that under the assumption of independent, identically distributed gaussian errors, i.e.

$$\varepsilon_i \sim \mathcal{N}(0, \sigma^2), \quad \sigma^2 < \infty, \quad \forall i$$

the MSE is exactly the MLE. While we manage to achieve this correspondence under Gaussian i.i.d. noise, MLE optimality alone does not guarantee good test risk performance when these assumptions break down, as the following examples illustrate.

Moreover, the next result characterizes how this estimator θ_{MSE} performs compared to other linear estimators:

Theorem 1.1.1 (Gauss-Markov). *Consider the model:*

$$Y_i = \beta^T X_i + \varepsilon_i$$

and suppose $\mathbb{E}[\varepsilon_i] = 0$, $\text{Var}[\varepsilon_i] = \sigma^2 < \infty$ and $\text{Cov}(\varepsilon_i, \varepsilon_j) = 0$, $\forall i \neq j$.

Then the OLS estimator is the Best Linear Unbiased Estimator (BLUE) of β .

1.2 Non-iid noise models

However, if the noise is non-i.i.d. and/or heavy-tailed, the use of the MSE loss can prove problematic, either introducing bias or letting the outliers have an uncontrolled influence on the forecast.

We will present here some relevant examples highlighting this fact, which motivate using more robust regression methods.

Example (Correlated Gaussian Noise). *Consider the same linear model as before, but this time with correlated gaussian noise:*

$$\varepsilon_{n \times 1} \sim \mathcal{N}(0, \Sigma)$$

*Here, θ_{MSE} is still **consistent**, however, it has a problem of efficiency: it is no longer the BLUE. Instead, we will have to reweight our objective to retrieve an optimal estimator in that sense. In this case, one can show that θ_{BLUE} is retrieved by minimizing the generalized least squares objective:*

$$\hat{\theta}_{BLUE} = \hat{\theta}_{GLS} = \underset{\theta}{\operatorname{argmin}} (Y - f_{\theta}(X))^T \Sigma^{-1} (Y - f_{\theta}(X))$$

However, in non gaussian settings, especially when the distribution is heavy-tailed, we cannot even guarantee consistency of the estimator.

Example (Cauchy Noise). Under Cauchy noise, a regime where outliers can happen with non-negligible probability, a new problem related to the MSE arises. Indeed, the outliers have far too strong of an influence on the fit. In order to formalize the argument, we will introduce the following notion of **Influence** of an estimator:

Definition 1.2.1 (Influence). The influence function of an estimator $\hat{\theta}$ defined by a statistical functional T at a contamination point y with respect to distribution F is:

$$\text{IF}(y; T, F) = \lim_{\epsilon \rightarrow 0} \frac{T((1-\epsilon)F + \epsilon\delta_y) - T(F)}{\epsilon} = \left. \frac{\partial}{\partial \epsilon} T((1-\epsilon)F + \epsilon\delta_y) \right|_{\epsilon=0} \quad (1)$$

where:

- $T : \mathcal{F} \rightarrow \Theta$ is the functional such that $\hat{\theta} = T(F_n)$ for the empirical measure F_n
- δ_y is the Dirac point mass at y
- $(1-\epsilon)F + \epsilon\delta_y$ is the **contaminated distribution**, i.e. a mixture placing mass ϵ on the outlier y and $(1-\epsilon)$ on the true distribution F

An estimator is called **B-robust** if

$$\sup_y \|\text{IF}(y; T, F)\| < \infty \quad (2)$$

i.e. the influence function is bounded over all possible contamination points y .

One can show that in the case of $\hat{\theta}_{\text{MSE}}$, and given the contribution $l(r) = r^2$ of a single outlier

$$\text{IF}(y; T_{\text{MSE}}, F) \propto \psi(r) := l'(r) = 2r \xrightarrow{|r| \rightarrow \infty} \infty$$

Given that a Cauchy random variable doesn't have a first or second moment, we can't even use the LLN to guarantee some kind of asymptotic convergence of the estimator.

Example (Covariate Shift). Consider a supervised regression with the true model being $y = f^*(x) + \varepsilon$, where f^* is unknown and ε is noise. We observe training pairs $(x_i^{\text{tr}}, y_i^{\text{tr}})_{i=1}^{n_{\text{tr}}} \sim p_{\text{tr}}(x)p(y|x)$, and evaluate on test pairs $(x^{\text{te}}, y^{\text{te}}) \sim p_{\text{te}}(x)p(y|x)$. The goal is to minimize the test risk $R_{\text{te}}(f) = \mathbb{E}_{x \sim p_{\text{te}}, y|x}[\ell(f(x), y)]$, $\ell(\hat{y}, y) = (\hat{y} - y)^2$. ERM instead minimizes the empirical training risk

$$\hat{R}_{\text{tr}}(f) = \frac{1}{n_{\text{tr}}} \sum_{i=1}^{n_{\text{tr}}} \ell(f(x_i^{\text{tr}}), y_i^{\text{tr}}),$$

which targets the population quantity $R_{\text{tr}}(f) = \mathbb{E}_{x \sim p_{\text{tr}}, y|x}[\ell(f(x), y)]$. When $p_{\text{tr}}(x) = p_{\text{te}}(x)$, this is aligned with the test objective. Under **covariate shift**, i.e. in the case where

$$p_{\text{tr}}(x) \neq p_{\text{te}}(x) \quad \text{while} \quad p_{\text{tr}}(y|x) = p_{\text{te}}(y|x).$$

Then ERM prioritizes accuracy where training inputs are frequent, even if those regions matter little at test time; the minimizers of R_{tr} and R_{te} can differ substantially, especially with a misspecified model class.

We will then use an **Importance-weighted** ERM in order to mitigate this effect.

Example (Batch Level Noise). Consider now additional perturbations at the batch level:

$$\tilde{Y}_i = Y_i + \varepsilon_i + b_p, \quad i \in I_p, \quad b_p \sim \mathcal{N}(0, \sigma_p^2)$$

For some model f_θ , the residual on batch p becomes:

$$\tilde{Y}_i - f_\theta(X_i) = \varepsilon_i + b_p \quad \text{for all } i \in I_p$$

The MSE gradient with respect to θ on batch p is then:

$$\nabla_\theta \mathcal{L}_{\text{MSE}} = -\frac{2}{B} \sum_{i \in I_p} (\tilde{Y}_i - f_\theta(X_i)) \nabla_\theta f_\theta(X_i)$$

At optimum $\theta = \theta^*$, we have $f_{\theta^*}(X_i) = Y_i$ so the residual reduces to $\varepsilon_i + b_p$, and the gradient becomes

$$\nabla_{\theta^*} \mathcal{L}_{\text{MSE}} = -\frac{2}{B} \sum_{i \in I_p} (\varepsilon_i + b_p) \nabla_{\theta^*} f_{\theta^*}(X_i)$$

Since b_p is constant within the batch, it does not cancel in the sum and acts as a systematic bias on the gradient direction, pushing f_θ away from f^* by an amount proportional to b_p .

1.3 WDRL vs WBM (Dakhmouche et al. 2025)

1.3.1 WDRL fails for non-convex/non-lipschitz model.

Today, many robust regression methods have been developed to tackle the issue of more global forms of noise than the classical i.i.d. Gaussian setting, with one of the most popular being the **Wasserstein Distributionally robust learning** (WDRL) paradigm.

However, as presented in [Dakhmouche et al., 2025], this paradigm can prove itself to be too restrictive for a Neural Network setting.

Indeed, for the loss function to be tractable, they have proposed the following two frameworks:

1. l_θ is a finite maximum of concave functions
2. l_θ is Lipschitz continuous

In the general case, we cannot guarantee to satisfy these conditions for a neural network. Indeed, neural networks are non-convex in their parameters by nature, and enforcing Lipschitzness impairs the expressive power of the Neural Networks.

Furthermore, and even when using this explicit expression of the loss function regardless of the fact that equality holds, for a Navier-Stokes operator with heavy-tailed noise (30% of corrupted data using Cauchy Noise), [Dakhmouche et al., 2025] found that the WDRL paradigm performs even worse than the standard MSE approach.

1.3.2 Wasserstein Batch Matching (WBM)

[Dakhmouche et al., 2025] propose a different and more suitable approach.

Given noisy outputs $\tilde{Y}_i = Y_i + \varepsilon_i$ and a model f_θ , the main idea is to relax the usual 1:1 correspondence between inputs and outputs. This is motivated by the fact the observed response can already deviate substantially from the true output, making pointwise correspondence unreliable. However, allowing features to match outputs in a distributional sense leads to increased robustness.

Thus, given a partition of the training data $\{(X_i, \tilde{Y}_i)\}_{i=1}^n$ into P batches (denoting $\{I_p\}_{p=1}^P$ for indices) define the empirical measures as

$$\mu_p^{\text{pred}} = \frac{1}{B} \sum_{i \in I_p} \delta_{f_\theta(X_i)} \quad \text{and} \quad \mu_p^{\text{obs}} = \frac{1}{B} \sum_{i \in I_p} \delta_{\tilde{Y}_i}$$

In this setting, the regression is formulated as follows:

$$\hat{\theta}_{WBM} \in \underset{\theta \in \mathbb{R}^d}{\operatorname{argmin}} \sum_{p=1}^P \mathcal{W}_2^2(\mu_p^{\text{pred}}, \mu_p^{\text{obs}})$$

Concretely, this Wasserstein distance for empirical measures is expressed as:

$$\mathcal{W}_2^2(\mu_p^{\text{pred}}, \mu_p^{\text{obs}}) = \min_{P \in \mathcal{C}_B} \sum_{i,j \in I_p} P_{ij} \left\| \tilde{Y}_i - f_\theta(X_j) \right\|^2$$

with $\mathcal{C}_B$ being the set of doubly-stochastic matrices. As we will investigate in this project, this relaxation of the pointwise correspondence and optimizing over the coupling matrices instead reduces sensitivity to corrupted correspondences.

2 First Experiments

2.1 WBM vs MSE without shuffling: fitting 1D function

We will first try to fit a simple $\mathbb{R}$ to $\mathbb{R}$ function, namely $x \mapsto \operatorname{sinc}(x)$ using a basic MLP. We first generate the true model: given uniformly generated inputs $\{X_i\}_{i=1}^n$, the true model reads

$$Y_i = \operatorname{sinc}(X_i)$$

that we then perturb using multivariate gaussian noise

$$Y = \operatorname{sinc}(X) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \Sigma)$$

with exponential decay, i.e. $\Sigma_{ij} = \sigma^2 \rho^{|i-j|}$ (Figure 1).

We will then apply the regression using, first, the classical MSE loss (`nn.MSELoss()`). This unsurprisingly leads to a decent fit of the function (Figure 1).

We then implement the WBM loss as follows and run the experiment again (Figure 1).

```
import ot

def wbm_loss(y_pred, y_obs):
    B = y_pred.shape[0]
    a = torch.ones(B, device=y_pred.device) / B
    b = torch.ones(B, device=y_obs.device) / B
    M = torch.cdist(y_obs, y_pred, p=2)**2
    loss = ot.sinkhorn2(a, b, M, reg=0.1, numItermax=100)
    return loss
```

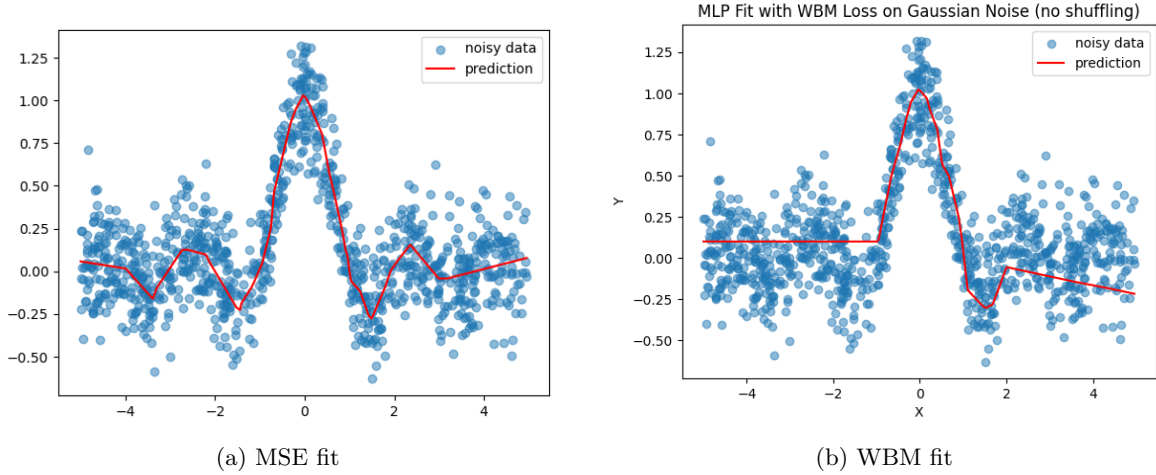


Figure 1

In this setting, in a similar number of epochs and with much higher computation time (mainly caused by the computation of the Sinkhorn algorithm in the WBM loss), we end up with a significantly worse fit (Figure 1, Figure 2). This is to be expected, since the pointwise MSE loss is exactly adapted to this type of simple setting. But as explained earlier, this setting is usually not what we witness in practical settings. In the following experiment, we will break this 1:1 correspondence in the data to highlight how that affects the standard framework.

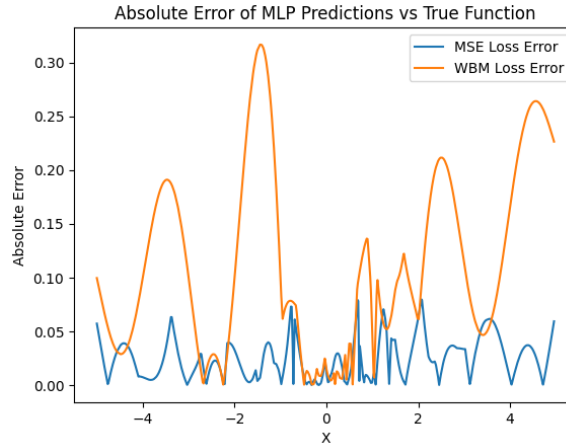


Figure 2

2.2 WBM vs MSE with shuffling: fitting 1D function

We now break the pointwise input-output correspondence, at batch level. More specifically, given $\tau \in \text{Perm}(\llbracket 1, n_b \rrbracket)$, with n_b the size of some batch b , our permuted dataset writes as

$$\tilde{D}_{n_b} = \{(X_i, Y_{\tau(i)})_{i=1}^{n_b}\}$$

In addition, we add a heavier-tailed noise.

We would expect this to completely break the standard MSE estimation. Indeed, the shuffling completely deletes any type of structural pointwise input/output relationship at batch level.

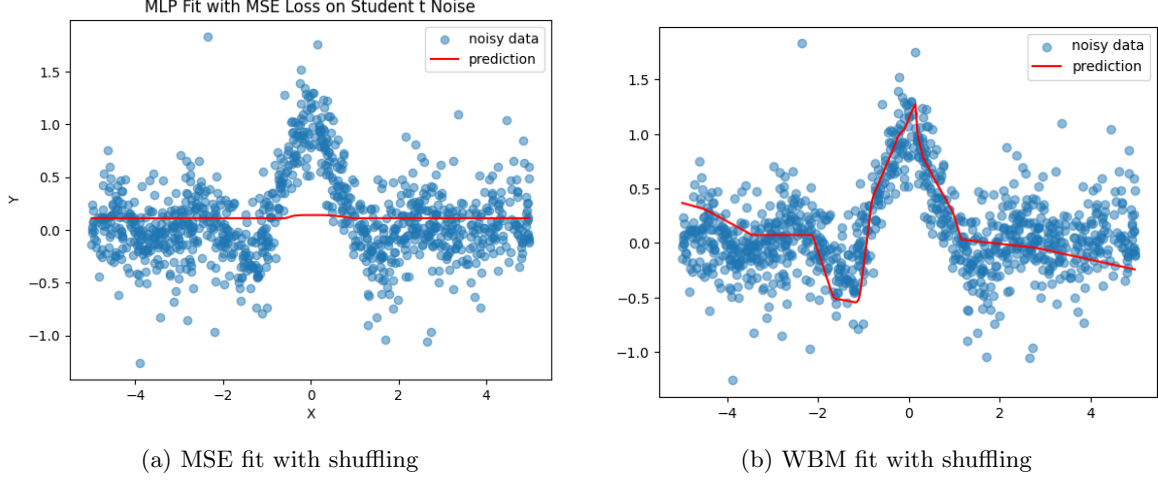


Figure 3

And our experiment confirms our intuition: the model almost fits a straight line around the mean value of the sinc function.

It is in this type of setting that the Wasserstein distributional robustness shines. Even with the shuffling, its flexibility to the input-output correspondence allows it to learn the general shape of the function way better (Figure 3b, Figure 4).

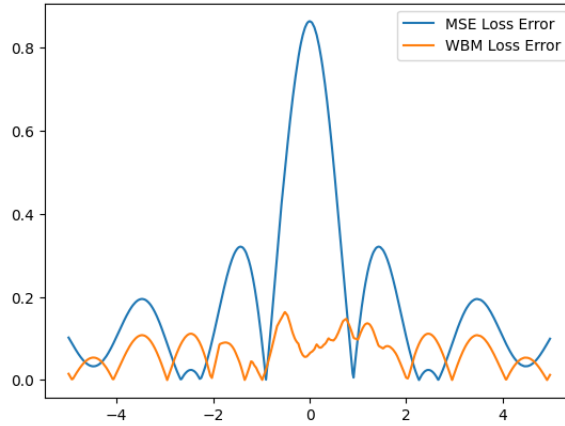


Figure 4: MSE vs WBM error

2.3 WBM vs MSE with shuffling: fitting 2D surface

This performance discrepancy is highlighted even further in the 2D case. Here, we will try to fit the following multivariate function.

$$f : (x, y) \mapsto \sin(\pi x) \cos(\pi y)$$

we will then perturb the observation using gaussian noise:

$$Z = f(X, Y) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.1)$$

then apply the previous input-output shuffling.

The MLP with MSE loss severely underfits in this case (Figure 5):

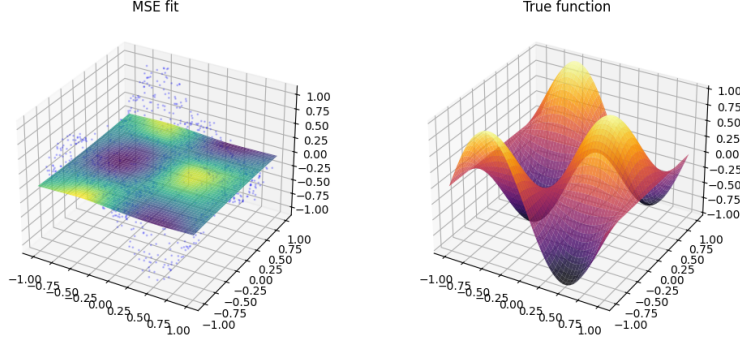


Figure 5: MSE fit vs True function f

The WBM fit, on the other hand, recovers the actual shape of the surface (Figure 6), capturing the alternating lobes and sign changes of $\sin(\pi x) \cos(\pi y)$ that the MSE fit completely misses.

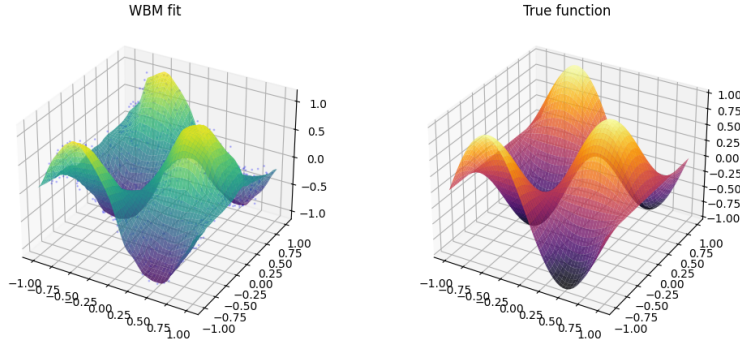


Figure 6: WBM fit vs True function f

And monitoring the absolute error confirms our intuition: the MSE framework essentially doesn't pick up on the shape of the function at all, whereas the WBM trained MLP manages to learn the shape of the surface.

Informally, a single batch only gives range information. But we have thousands of batches covering the whole domain simultaneously. The intersection of constraints (the model must match output distributions at every batch, everywhere on the domain) leaves very little room for alternative solutions. A "wrong" surface would have to match the output distribution of the function everywhere simultaneously, which is essentially impossible for a smooth function.

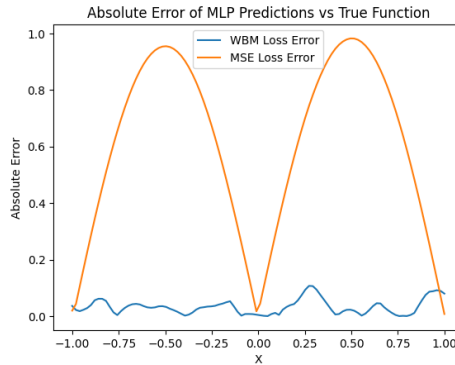


Figure 7: Absolute Error of first coordinate

2.4 Takeaway

These toy experiments practically motivate the use of the WBM framework in non iid noise regimes, especially when the 1:1 correspondence between inputs and outputs is severely broken. Considering this noise type is far from being a vain endeavor: loss of correspondence can arise in RNA measurements in cells, where the measurement procedure (scRNA-seq) by itself causes the destruction of the cell. This is also relevant when we are subject to privacy constraints on the data; we should then rely on empirical batch distribution correspondences.

3 Interlude on Influence

One could reasonably argue that the aforementioned measure of robustness by means of the influence function is an arbitrary assessment of performance. However, it is the state-of-the-art metric by which we monitor the sensitivity of an estimator to outliers.

3.1 Influence function of $\hat{\theta}_{\text{WBM}}$

Recall the definition of Influence we introduced earlier (1.2.1):

Definition 3.1.1 (Influence). *The influence function of an estimator $\hat{\theta}$ defined by a statistical functional T at a contamination point y with respect to distribution F is:*

$$\begin{aligned} \text{IF}(y; T, F) &= \lim_{\varepsilon \rightarrow 0} \frac{T((1 - \varepsilon)F + \varepsilon\delta_y) - T(F)}{\varepsilon} \\ &= \left. \frac{\partial}{\partial \varepsilon} T((1 - \varepsilon)F + \varepsilon\delta_y) \right|_{\varepsilon=0} \end{aligned}$$

as well the definition of our estimator:

$$\hat{\theta}_{\text{WBM}} = \underset{\theta \in \mathbb{R}^d}{\operatorname{argmin}} \sum_{p=1}^P \mathcal{W}_2^2(\mu_p^{\text{pred}}, \mu_p^{\text{obs}})$$

We first aim to derive the score equation for this estimator, i.e.

$$\nabla_{\theta} \underset{\theta \in \mathbb{R}^d}{\operatorname{argmin}} \sum_{p=1}^P \mathcal{W}_2^2(\mu_p^{\text{pred}}, \mu_p^{\text{obs}}) = 0$$

which, by Danskin's theorem, is of the form

$$\sum_{p=1}^P \sum_{i,j \in I_p} P_{ij}^*(\theta) \nabla_{\theta} \left\| \tilde{Y}_i - f_{\theta}(X_j) \right\|^2 = 0$$

We then apply the contamination $(1 - \varepsilon)F + \varepsilon\delta_y$ to the empirical distribution of the observed batch, which then becomes

$$\mu_p^{\text{obs}, \varepsilon} = \frac{1 - \varepsilon}{B} \sum_{i \in I_p} \delta_{\tilde{Y}_i} + \varepsilon\delta_y$$

thus, we can define the function of the perturbation parameter

$$\theta(\varepsilon) = T_{\text{WBM}}((1 - \varepsilon)F + \varepsilon\delta_y)$$

such that

$$\text{IF}(y; T_{\text{WBM}}, F) = \lim_{\varepsilon \rightarrow 0} \frac{T((1 - \varepsilon)F + \varepsilon\delta_y) - T(F)}{\varepsilon} = \theta'(0)$$

Now, the score equation for the perturbed loss becomes

$$\sum_{p=1}^P \sum_{i,j \in I_p} P_{ij}^*(\theta(\varepsilon), \varepsilon) \nabla_{\theta} \left\| \tilde{Y}_i^{(\varepsilon)} - f_{\theta(\varepsilon)}(X_j) \right\|^2 = 0, \quad \forall \varepsilon$$

In other terms, $\theta(\varepsilon)$ is our estimator for the perturbed distribution. We are going to differentiate this equation again at $\varepsilon = 0$, in order to retrieve $\theta'(0)$.

Proposition 3.1.1. *Under standard regularity assumptions (non-degenerate optimum, f_θ differentiable in θ), the influence function of $\hat{\theta}_{\text{WBM}}$ is given by:*

$$\text{IF}(y; T_{\text{WBM}}, F) = -A^{-1}B(y)$$

where A and $B(y)$ are defined explicitly in the proof.

Proof. We argue by the implicit function theorem, the vector analogue of the scalar identity $\frac{d}{dx}F(x, y(x)) = \partial_1 F + y'(x) \partial_2 F = 0 \Rightarrow y'(x) = -\partial_1 F / \partial_2 F$.

Write $r_{ij}(\theta) := \|\tilde{Y}_i - f_\theta(X_j)\|^2$ and let $\theta(\varepsilon) := T_{\text{WBM}}((1 - \varepsilon)F + \varepsilon\delta_y)$, so that $\text{IF}(y; T_{\text{WBM}}, F) = \theta'(0)$. Because the loss is the entropically regularized transport cost (Sinkhorn), the optimal coupling $P^*(\theta, \varepsilon)$ is unique and continuously differentiable in (θ, ε) near $\varepsilon = 0$; this justifies differentiating P^* below. By Danskin's theorem (envelope form, optimal coupling held fixed), the first-order condition for the contaminated problem reads

$$G(\theta(\varepsilon), \varepsilon) := \sum_{p=1}^P \sum_{i,j \in I_p} P_{ij}^*(\theta(\varepsilon), \varepsilon) \nabla_\theta r_{ij}(\theta(\varepsilon)) + \sum_{p=1}^P \sum_{j \in I_p} P_{yj}^*(\theta(\varepsilon), \varepsilon) \nabla_\theta \|y - f_{\theta(\varepsilon)}(X_j)\|^2 = 0, \quad \forall \varepsilon,$$

where the second sum accounts for the contamination atom y (zero mass at $\varepsilon = 0$).

Since $G(\theta(\varepsilon), \varepsilon) \equiv 0$, total differentiation in ε gives

$$\partial_\theta G(\theta, 0) \theta'(0) + \partial_\varepsilon G(\theta, 0) = 0, \quad \text{hence} \quad \theta'(0) = -[\partial_\theta G(\theta, 0)]^{-1} \partial_\varepsilon G(\theta, 0),$$

provided $\partial_\theta G(\theta, 0)$ is invertible. It remains to identify the two factors. Differentiating a generic summand $P_{ij}^* \nabla_\theta r_{ij}$ by the product rule and the chain rule, at $(\theta, 0)$,

$$\frac{d}{d\varepsilon} [P_{ij}^* \nabla_\theta r_{ij}] = \underbrace{[(\partial_\theta P_{ij}^*)^\top \theta'(0) + \partial_\varepsilon P_{ij}^*] \nabla_\theta r_{ij}}_{\text{coupling varies}} + \underbrace{P_{ij}^* H_{ij} \theta'(0)}_{\text{residual varies}}, \quad H_{ij} := \nabla_\theta^2 r_{ij}.$$

Collecting all terms proportional to $\theta'(0)$ yields $A := \partial_\theta G(\theta, 0)$, and the explicit ε -terms (including the contamination atom, whose weight derivative at 0 is the routing $\pi_j(y) := \partial_\varepsilon P_{yj}^*|_0$) yield $B(y) := \partial_\varepsilon G(\theta, 0)$:

$$A = \sum_{p=1}^P \sum_{i,j \in I_p} \left[\nabla_\theta r_{ij} (\partial_\theta P_{ij}^*)^\top + P_{ij}^* H_{ij} \right] \in \mathbb{R}^{d \times d},$$

$$B(y) = \underbrace{\sum_{p=1}^P \sum_{i,j \in I_p} \partial_\varepsilon P_{ij}^*|_{(\theta,0)} \nabla_\theta r_{ij}}_{\text{reweighting of existing atoms}} + \underbrace{\sum_{p=1}^P \sum_{j \in I_p} \pi_j(y) \nabla_\theta \|y - f_\theta(X_j)\|^2}_{\text{contribution of the contamination atom } y} \in \mathbb{R}^d.$$

Substituting gives the claim:

$$\boxed{\text{IF}(y; T_{\text{WBM}}, F) = \theta'(0) = -A^{-1}B(y)}$$

where A depends only on the unperturbed problem, while $B(y)$ encodes the sensitivity to the contamination point y , entirely through the optimal coupling and the contamination term's gradient. $\square$

3.2 The linear case

So far, this expression for the influence function holds in full generality, but it doesn't tell us much about whether $\hat{\theta}_{\text{WBM}}$ is actually B-robust, since everything is hidden inside the coupling-sensitivity term $B(y)$. To make this concrete, we now restrict ourselves to the linear case, where the coupling becomes explicit and we can determine exactly how $B(y)$ behaves as the contamination point y grows.

Proposition 3.2.1 (Linear case: failure of B-robustness). *Consider the linear model $f_\theta(X) = \theta^\top X$, with $\theta, X_j \in \mathbb{R}^d$ and scalar outputs, a squared Euclidean ground cost and a fixed Sinkhorn regularization $\lambda > 0$. Assume the design is non-degenerate, so that the matrix A from Proposition 3.1.1 is invertible, and that the predictions $\hat{y}_j := \theta^\top X_j$ are not all equal. Then the influence function of $\hat{\theta}_{\text{WBM}}$ grows linearly in the contamination point:*

$$\|\text{IF}(y; T_{\text{WBM}}, F)\| = \Theta(|y|) \quad \text{as } |y| \rightarrow \infty.$$

In particular $\sup_y \|\text{IF}(y; T_{\text{WBM}}, F)\| = \infty$, so $\hat{\theta}_{\text{WBM}}$ is not B-robust.

Proof. We simply specialize the decomposition $\text{IF}(y) = -A^{-1}B(y)$ from Proposition 3.1.1 to the linear model. Here

$$r_{ij}(\theta) = (\tilde{Y}_i - \theta^\top X_j)^2, \quad \nabla_\theta r_{ij} = -2(\tilde{Y}_i - \theta^\top X_j) X_j, \quad H_{ij} := \nabla_\theta^2 r_{ij} = 2 X_j X_j^\top,$$

so the Hessian is constant in θ and doesn't depend on y . Since A is built only from the unperturbed coupling, its θ -derivative, and these H_{ij} , it does not depend on y either, and by assumption it is a fixed invertible matrix. So all the y -dependence of $\text{IF}(y)$ sits in $B(y)$, and it is enough to show $\|B(y)\| = \Theta(|y|)$.

Now, the key thing to notice is that the contamination cannot be ignored. Adding the outlier y to the observed marginal gives $a^\varepsilon = (\frac{1-\varepsilon}{B}, \dots, \frac{1-\varepsilon}{B}, \varepsilon)$, where the last entry is y . Since the row sums of the coupling must match the marginal, $\sum_j P_{yj}^*(\varepsilon) = \varepsilon$, and differentiating at $\varepsilon = 0$ gives

$$\sum_j \pi_j(y) = 1, \quad \pi_j(y) := \partial_\varepsilon P_{yj}^*|_0.$$

In other terms, $\pi(y)$ is a probability distribution over the prediction atoms, and from the Gibbs form of the entropic coupling it is the softmax

$$\pi_j(y) = \frac{v_j e^{-\|y - \hat{y}_j\|^2/\lambda}}{\sum_k v_k e^{-\|y - \hat{y}_k\|^2/\lambda}},$$

with v_k the fixed unperturbed scalings. The point is that $\pi(y)$ never vanishes: the outlier is always fully transported somewhere, it is never down-weighted to zero.

It remains to control the two terms of $B(y)$. The first one, $\sum_p \sum_{i,j} \partial_\varepsilon P_{ij}^*|_0 \nabla_\theta r_{ij}$, only depends on y through the way the outlier perturbs the coupling among the existing atoms. Cancelling the common $|y|^2$ in the softmax logits, the effective logit of atom j is $\log v_j + (2y\hat{y}_j - \hat{y}_j^2)/\lambda$, so as $|y| \rightarrow \infty$ the outlier's mass piles onto a single boundary atom $j^*(y)$ and its effect on the rest of the coupling settles to a finite limit. More precisely, expand the squared cost in each exponent (the outputs are scalar, so $\|y - \hat{y}_j\|^2 = (y - \hat{y}_j)^2$):

$$(y - \hat{y}_j)^2 = y^2 - 2y\hat{y}_j + \hat{y}_j^2,$$

where the term y^2 is common to every atom j . Factoring it out of each exponential,

$$e^{-(y - \hat{y}_j)^2/\lambda} = e^{-y^2/\lambda} e^{(2y\hat{y}_j - \hat{y}_j^2)/\lambda},$$

and since the factor $e^{-y^2/\lambda}$ appears identically in the numerator and in every summand of the denominator, it cancels:

$$\pi_j(y) = \frac{v_j e^{-(y - \hat{y}_j)^2/\lambda}}{\sum_k v_k e^{-(y - \hat{y}_k)^2/\lambda}} = \frac{v_j e^{(2y\hat{y}_j - \hat{y}_j^2)/\lambda}}{\sum_k v_k e^{(2y\hat{y}_k - \hat{y}_k^2)/\lambda}}.$$

The only remaining y -dependence in the logit of atom j is the term $2y\hat{y}_j/\lambda$, which is linear in y with slope $\hat{y}_j$. Hence the gap between the logits of two atoms j and k is

$$\frac{2y(\hat{y}_j - \hat{y}_k)}{\lambda} + \text{const},$$

and as $|y| \rightarrow \infty$ this gap diverges in favour of the atom with the most extreme $\hat{y}_j$ in the direction of y . The softmax therefore concentrates on a single boundary atom $j^*(y)$. Hence this first term stays bounded, it is $O(1)$.

The second reads

$$\sum_j \pi_j(y) \nabla_\theta \|y - \theta^\top X_j\|^2 = -2 \sum_j \pi_j(y) (y - \hat{y}_j) X_j,$$

and using $\sum_j \pi_j(y) = 1$ we immediately get the upper bound

$$\left\| \sum_j \pi_j(y) (y - \hat{y}_j) X_j \right\| \leq \max_j \|X_j\| (|y| + \max_j |\hat{y}_j|) = O(|y|).$$

For the lower bound, the softmax concentrates on $j^*(y)$ as $|y| \rightarrow \infty$, so this term behaves like $-2(y - \hat{y}_{j^*})X_{j^*} + o(|y|)$, whose norm grows like $2\|X_{j^*}\| |y|$. And since $X_{j^*} \neq 0$, this term diverges to infinity. Putting the two terms together, $\|B(y)\| = \Theta(|y|)$, and therefore $\|\text{IF}(y)\| = \Theta(|y|) \rightarrow \infty$. $\square$

Remark 3.2.1. *The reason behind this is structural. A redescending M-estimator is B-robust precisely because its influence weight $\psi(r) \rightarrow 0$ for large residuals: past some point the outlier is simply ignored. Which is not the case for WBM. Mass conservation in the optimal transport forces $\sum_j \pi_j(y) = 1$, so every observation, outlier or not, has to be matched to some prediction, and it keeps contributing a gradient that grows with $|y|$. In a sense, WBM never gives up on a corrupted point. This actually matches what we saw in the experiments: WBM is robust to corruption that leaves the batch output distribution unchanged (shuffling leaves μ_p^{obs} invariant, so the estimator doesn't even see it, even at full corruption), but fragile to corruption that moves this distribution, like the heavy-tailed Cauchy noise.*

4 WBM applied to operator learning

In this section, we will follow the implementation proposed by Dakhmouche et al., where CNOs have been used in order to learn the Navier-Stokes solution operator, and we will try to extend the experiments to different types of data corruption, giving a comprehensive L^1 vs WBM comparison.

4.1 Operator Learning: A quick overview

In what follows, we will attempt to give a quick introduction to operator learning, presenting the important concepts and the problem to solve. In order to do so, let's first recall the usual setting of a **Partial Differential Equation (PDE)**

$$\mathcal{L}u = f$$

for some operator $\mathcal{L}$ and functions $u \in \mathcal{U}$ and $f \in \mathcal{V}$, with $\mathcal{U}$ and $\mathcal{V}$ function spaces on a d-dimensional subset $\Omega \subset \mathbb{R}^d$. We can then define the "forward problem", i.e. finding an operator $\mathcal{A} : \mathcal{V} \rightarrow \mathcal{U}$ such that $\mathcal{A}(f) = u$. In other words, $\mathcal{A}$ can be seen as the "solution operator" of the PDE, or, informally, as the "inverse" of the operator $\mathcal{L}$. The main objective then becomes to approximate $\mathcal{A}$ by an operator $\hat{\mathcal{A}}$ such that, for all new data f' , we have $\mathcal{A}(f') \approx \hat{\mathcal{A}}(f')$. We then apply the classical ML framework for solving this type of problem, namely optimizing an ERM-like expression:

$$\min_{\theta \in \mathbb{R}^N} \sum_{(f,u) \in \text{data}} L(\hat{\mathcal{A}}(f; \theta), u)$$

with L some pre-defined loss.

It is worth noting that recovering the solution operator is usually challenging. However, the "forward problem" is usually well-posed, i.e.

- (1) there **exists** a solution to the problem
- (2) the solution is **unique**.
- (3) the solution **depends continuously** on the given data

Now, the data upon which we train the neural operator are approximate solutions from PDE solvers, which introduce discretisation and truncation errors. These errors are spatially correlated: rather than pointwise, the whole approximate solution $\tilde{u}^{(i)}$ is corrupted. This approximation error arises at the solver level, and can be written as:

$$\tilde{u}^{(i)} = u^{(i)} + b_p + \varepsilon_i, \quad p \in \mathcal{P}$$

with $\mathcal{P}$ an index set for the collection of PDE solvers, and, informally,

$$b_p := b_p(\text{scheme, mesh resolution, mesh size, } \dots)$$

This is very analogous to the notion of **batch-level noise** we introduced earlier, and it thus motivates the use of the **WBM** to handle this "solver-level noise".

4.2 Convolutional Neural Operators

4.2.1 Neural Operator

A neural operator can be, informally, seen as a continuous generalization of a neural network. Whereas neural networks learn mappings between finite dimensional discrete vector spaces, neural operators aim to approximate mappings between (possibly infinite dimensional) function spaces. Recall the well-known definition of a L-layer Neural network

$$\mathcal{N}(x) = \sigma(A_L(\cdots \sigma(A_1 x + b_1) \cdots) + b_L)$$

with A_i the weight matrices, b_i the bias vectors and σ the activation function. Imitating this structure, a layer of a neural operator can be defined as

$$u_{i+1} = \sigma \left(\int_{\Omega_i} K^{(i)}(x, y) u_i(y) dy + b_i(x) \right), \quad x \in \Omega_{i+1}$$

4.2.2 CNOs

In the scope of this experiment, we will consider the convolutional neural operator architecture, which, similarly to U-Net, operates on bandlimited functions via alternating upsampling and downsampling operators. Unlike an FNO architecture, this allows us more flexibility in that we can relax the periodicity assumption that isn't naturally suited to many PDE solutions (notably Navier-Stokes). More formally, it can be defined as a compositional mapping between functions:

$$\mathcal{G} : u \mapsto P(u) = v_0 \mapsto v_1 \mapsto \cdots v_L \mapsto Q(v_L) = \bar{u}$$

with

$$v_{l+1} = \mathcal{P}_l \circ \Sigma_l \circ \mathcal{K}_l(v_l)$$

where $\mathcal{K}_l$ is a convolution operator, Σ_l is the activation layer, and $\mathcal{P}_l$ is an up- or down-sampling operator. Then, P sends the input u into the space of bandlimited functions, and Q is a projection onto the output space [2].

4.3 Learning Navier Stokes

We follow Dakhmouche's experimental setup for operator learning on the Navier-Stokes equations, now systematically varying the corruption level $\rho \in \{0, 0.25, 0.50, 0.75, 1.00\}$, where ρ denotes the fraction of training outputs perturbed.

We use the existing CNO implementation by Raonic, and the relative L^1 as the baseline, following Raonic's CNO setup (CN02d_simplified/CN02d.py).

We also note that the operator-learning experiments use a slightly different realization of WBM than the one analysed in Section 3: here the batch matching is solved as an exact optimal assignment (Hungarian algorithm) with an L^1 ground cost, rather than the entropically regularized squared-cost (W_2^2 , Sinkhorn) version used in the low-dimensional experiments. The batch-matching principle is the same in both cases; only the coupling and the ground cost differ.

Corruption is applied at the batch level by randomly shuffling $\rho \cdot B$ outputs within each batch of size B :

```
def corrupt_batch(yb, rho):
    B = yb.shape[0]
    n_corrupt = int(rho * B)
    if n_corrupt == 0:
        return yb.clone()
    idx = torch.randperm(B)[:n_corrupt]
    shuffled = yb[idx][torch.randperm(n_corrupt)]
    yb_corrupt = yb.clone()
    yb_corrupt[idx] = shuffled
    return yb_corrupt
```

The CNO is then trained under each corruption regime using both the L^1 loss and the WBM framework. Table 1 reports the test relative L^1 errors, and Figure 8 visualises the comparison across corruption levels.

ρ	L^1	WBM
0.00	0.2136	0.1621
0.25	0.2269	0.2327
0.50	0.2752	0.2142
0.75	0.3463	0.1683
1.00	0.3508	0.3493

Table 1: Test relative L^1 error for CNO on Navier-Stokes under batch corruption rate ρ .

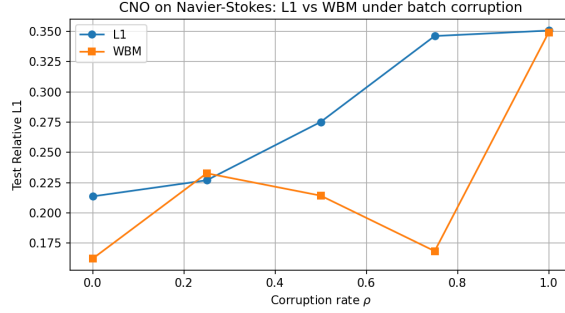


Figure 8: Test relative L^1 error under L^1 loss vs. WBM across corruption levels ρ .

The two columns behave very differently across ρ . The L^1 error degrades monotonically as corruption increases (from 0.2136 at $\rho = 0$ to 0.3508 at $\rho = 1$), which is expected: shuffling destroys the pointwise correspondence that L^1 relies on. The WBM column, on the other hand, stays essentially flat for $\rho \leq 0.75$, hovering around 0.16–0.23 with no clear trend in ρ . The single exception is $\rho = 1.00$ (0.3493): since within-batch shuffling leaves μ_p^{obs} invariant, the $\rho = 1.00$ error should coincide with $\rho = 0$ in expectation, so we interpret this value as single-run optimization noise rather than a real ρ -dependence. This is exactly what the invariance noted in Section 3 predicts: within-batch shuffling permutes the outputs of a batch but leaves the empirical measure μ_p^{obs} unchanged, so in expectation the WBM loss does not depend on ρ at all. The takeaway is not that WBM improves under corruption, but that it is *indifferent* to it, while L^1 steadily collapses towards WBM’s already-low level (most visibly at $\rho = 0.75$, where L^1 has degraded to 0.3463 while WBM sits at 0.1683). This advantage is further illustrated at the field level in Figure 9: under high corruption, the L^1 -trained model fails to capture vorticity dynamics, producing near-uniform predictions, whereas the WBM model recovers the qualitative structure of the vorticity field, including its curved geometry.

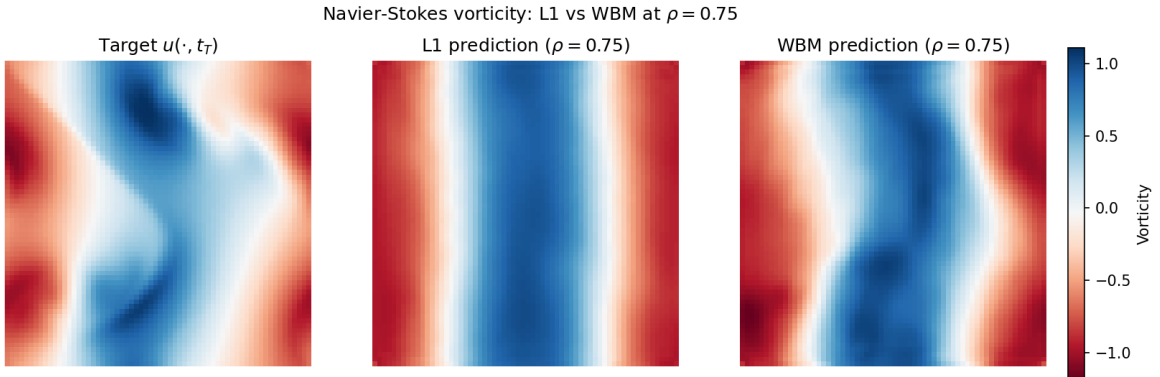


Figure 9: Predicted vorticity fields at high corruption: L^1 loss (left) vs. WBM (right).

Now, we follow the framework we established earlier, and establish a Cauchy Noise at Batch level. This time, we train for 100 epochs (double the previous budget) to make sure the comparison is not confounded by under-training, and we focus on the corruption levels $\rho \in \{0, 0.25, 0.75\}$. Here, the picture is different: WBM keeps its advantage at low and moderate corruption ($\rho = 0, 0.25$), but is overtaken by L^1 at $\rho = 0.75$ (0.308 vs. 0.206), as shown in Figures 10, 11 and 12. These are single-run

results, so the small non-monotonicities (such as the mild improvement from $\rho = 0$ to $\rho = 0.25$) should be read as run-to-run variation. The crossover at high corruption, however, is consistent with the non-B-robustness of $\hat{\theta}_{\text{WBM}}$ derived in Section 3: under heavy Cauchy contamination, mass conservation in the optimal transport forces every point to remain matched to a prediction, so WBM cannot discard the heavy Cauchy outliers regardless of the ground cost, and this lack of an outlier-bounding mechanism eventually outweighs its tolerance to broken correspondence.

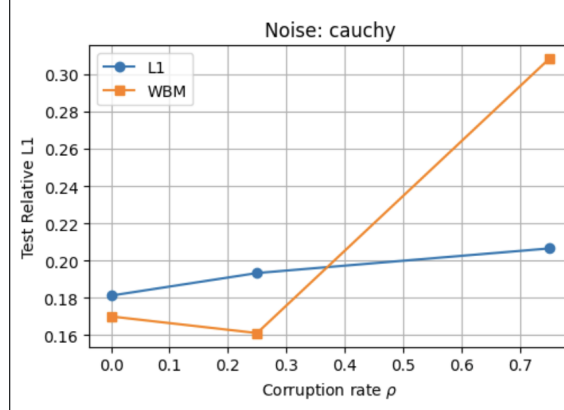


Figure 10: L^1 error comparison, Cauchy noise.

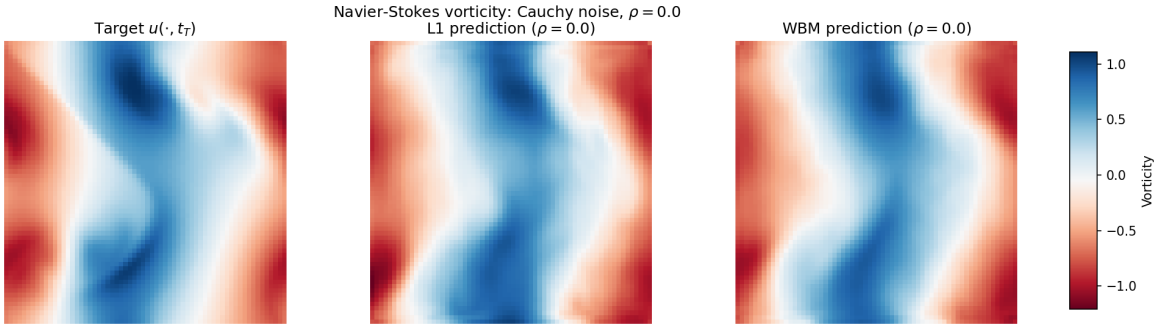


Figure 11: Vorticity predictions, 0% corruption.

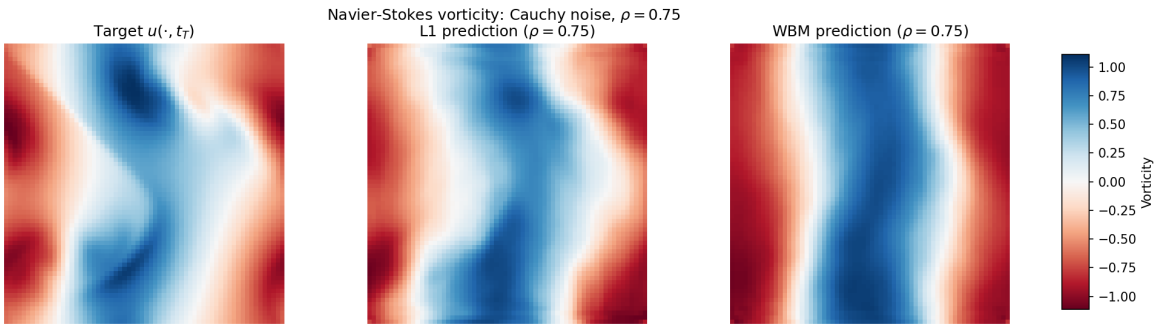


Figure 12: Vorticity predictions, 75% corruption.

Note that Figure 12 shows a representative sample; WBM still produces a visually plausible field, but its higher aggregate error at $\rho = 0.75$ reflects worse performance averaged over the test set.

5 Conclusion

The assumption of i.i.d. noise underpins classical regression, yet it is precisely the assumption that fails in practice. We studied the Wasserstein Batch Matching framework of [1] as a response, and the

picture that emerges is very interesting: WBM replaces the pointwise input-output correspondence with a distributional batch-level matching, and this single change is what makes it robust exactly where standard regression breaks, namely when the 1:1 correspondence is lost.

Our main contribution is to explain and put into practice this phenomenon. We derived the influence function of the WBM estimator and showed, in the linear case, that it is not B-robust: its influence grows linearly in the contamination point. The reason is structural. Optimal-transport mass conservation forces every observation, outliers included, to be matched to a prediction, so WBM can never discard a corrupted point the way a redescending M-estimator would. Its strength is therefore not bounded influence, but tolerance to broken correspondence, and that distinction turns out to predict everything else.

However, the WBM framework remains of great interest, and both the theory and experiment highlight this idea. On the low-dimensional targets, MSE collapses to the mean under shuffling while WBM recovers the global shape. On the Navier-Stokes operator, WBM stays reasonably low in the corruption rate under shuffling while L^1 degrades steadily, since shuffling leaves the batch output distribution untouched. And under heavy-tailed Cauchy noise the advantage reverses only at high corruption: WBM retains its edge at low and moderate contamination ($\rho = 0, 0.25$), consistent with the heavy-tail tolerance that motivates it in [1], and is overtaken only once contamination is severe, where mass conservation prevents it from discarding outliers.

References

- [1] Ramzi Dakhmouche, Ivan Lunati, and Hossein Gorji. Noise tolerance of distributionally robust learning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, Vancouver, Canada, 2025.
- [2] Bogdan Raonić, Roberto Molinaro, Tim De Ryck, Tobias Rohner, Francesca Bartolucci, Rima Alaifari, Siddhartha Mishra, and Emmanuel de Bézenac. Convolutional neural operators for robust and accurate learning of pdes. In *Advances in Neural Information Processing Systems*, 2023.